**Q1. What is Python and why is it called an interpreted language?**

**Answer:** Python is a high-level, general-purpose programming language. It executes code line by line at runtime using an interpreter, without compiling the whole program first.

---

**Q2. What is the difference between a variable and a constant in Python?**

**Answer:** A variable is a name that stores a value which can be changed anytime. Python has no built-in constant type, but by convention, constants are written in ALL_CAPS to signal they should not be changed.

---

**Q3. What are the built-in data types available in Python?**

**Answer:** Python has int, float, complex, str, bool, list, tuple, set, frozenset, dict, and NoneType as built-in data types. They are divided into numeric, sequence, set, and mapping categories.

---

**Q4. What is the difference between an integer and a float in Python?**

**Answer:** An integer is a whole number without a decimal point like 5. A float is a number with a decimal point like 5.0. Both are numeric types but stored differently in memory.

---

**Q5. What is a string and how is it represented in Python?**

**Answer:** A string is a sequence of characters enclosed in single, double, or triple quotes. Example: 'hello', "world", or """multiline""". Strings are immutable in Python.

---

**Q6. What is the difference between single quotes and double quotes in Python strings?**

**Answer:** There is no functional difference between them in Python. Both define a string. The choice matters only when the string itself contains a quote character, to avoid escape characters.

---

**Q7. What is a boolean and what are its possible values in Python?**

**Answer:** A boolean is a data type that holds only two values: True or False. It is a subclass of int in Python, where True equals 1 and False equals 0.

---

**Q8. What is a list and how is it different from a regular array?**

**Answer:** A list is an ordered, mutable collection that can hold elements of different data types. Unlike a regular array, it does not require all elements to be of the same type.

---

**Q9. What is a tuple and why is it considered immutable?**

**Answer:** A tuple is an ordered collection defined with parentheses like (1, 2, 3). It is immutable because once created, its elements cannot be added, removed, or changed.

---

**Q10. What is a dictionary and how does it store data internally?**

**Answer:** A dictionary stores data as key-value pairs like {'name': 'Alice'}. Internally it uses a hash table, which allows fast lookup of values using their keys.

---

**Q11. What is a set and what makes it different from a list?**

**Answer:** A set is an unordered collection of unique elements defined with {}. Unlike a list, it does not allow duplicate values and does not maintain insertion order.

---

**Q12. What is None and what type does it belong to in Python?**

**Answer:** None represents the absence of a value in Python. Its type is NoneType and it is commonly used as a default return value when a function returns nothing.

---

**Q13. What is an operator and what are the different types of operators in Python?**

**Answer:** An operator is a symbol that performs an operation on values. Python has arithmetic (+, -), comparison (==, >), logical (and, or), assignment (=), bitwise, and identity operators.

---

**Q14. What is a function and why do we use it in Python?**

**Answer:** A function is a reusable block of code defined with the def keyword. We use functions to avoid repeating code, improve readability, and break a program into smaller logical pieces.

---

**Q15. What is a module and how is it different from a script?**

**Answer:** A module is a Python file containing functions, classes, or variables meant to be imported and reused. A script is a Python file meant to be run directly as a program.

---

**Q16. What is a package and how is it structured in Python?**

**Answer:** A package is a directory containing multiple modules and a special __init__.py file. It allows grouping related modules together and importing them using dot notation like package. module.

---

**Q17. What is indentation and why is it mandatory in Python?**

**Answer:** Indentation is the use of spaces or tabs at the beginning of a line to define a block of code. Python uses indentation instead of curly braces, so incorrect indentation causes an IndentationError.

---

**Q18. What is a single-line comment and a multi-line comment in Python?**

**Answer:** A single-line comment starts with #. A multi-line comment is written using triple quotes """..."""  or '''...''', though technically those are string literals used as comments by convention.

---

**Q19. What is a for loop and how does it iterate over a sequence?**

**Answer:** A for loop iterates over each element of a sequence like a list, tuple, or string one by one. It automatically moves to the next element until the sequence is exhausted.

---

**Q20. What is a while loop and when should it be used over a for loop?**

**Answer:** A while loop runs as long as a condition is True. It is preferred over for when the number of iterations is not known in advance, such as waiting for user input.

---

**Q21. What is an if statement and how does Python evaluate its condition?**

**Answer:** An if statement checks a condition and runs its block only if the condition is True. Python evaluates the condition as a boolean, treating values like 0, None, and empty collections as False.

---

**Q22. What is the difference between else and elif in a conditional block?**

**Answer:** else runs when all previous conditions are False and takes no condition. elif is used to check an additional condition when the previous if was False, allowing multiple branches.

---

**Q23. What is a return statement and what happens when a function has no return?**

**Answer:** return sends a value back to the caller and exits the function. If a function has no return statement, it automatically returns None.

---

**Q24. What is the difference between an argument and a parameter in a function?**

**Answer:** A parameter is the variable name defined in the function definition. An argument is the actual value passed to the function when it is called.

---

**Q25. What is a default parameter and what happens if it is not passed during a call?**

**Answer:** A default parameter has a pre-assigned value in the function definition like def foo(x=10). If the caller does not pass that argument, the default value is used automatically.

---

**Q26. What is type casting and what is the difference between implicit and explicit casting?**

**Answer:** Type casting converts one data type to another. Implicit casting is done automatically by Python like adding int and float. Explicit casting is done manually using functions like int(), str(), or float().

---

**Q27. What is the input() function and what data type does it always return?**

**Answer:** input() reads data entered by the user from the keyboard. It always returns the value as a str regardless of what the user types, so you must cast it if you need a number.

---

**Q28. What is the print() function and what does the end and sep parameter do?**

**Answer:** print() outputs values to the console. sep defines the separator between multiple values (default is space), and end defines what is printed at the end (default is newline \n).

---

**Q29. What is len() and on which data types can it be used?**

**Answer:** len() returns the number of items in an object. It works on strings, lists, tuples, sets, dictionaries, and any other iterable that has a defined length.

---

**Q30. What is range() and what are its three possible arguments?**

**Answer:** range() generates a sequence of numbers. Its three arguments are start (default 0), stop (required, exclusive), and step (default 1). Example: range(1, 10, 2) gives 1, 3, 5, 7, 9.

---

**Q31. What is indexing and how does Python handle positive and negative indexing?**

**Answer:** Indexing accesses a specific element using its position. Positive indexing starts from 0 at the beginning, and negative indexing starts from -1 at the end of the sequence.

---

**Q32. What is slicing and what does the syntax [start:stop:step] mean?**

**Answer:** Slicing extracts a portion of a sequence. start is the beginning index, stop is the end index (exclusive), and step defines how many elements to skip. Example: a[0:5:2].

---

**Q33. What is string concatenation and what operator is used for it?**

**Answer:** String concatenation is joining two or more strings together. The + operator is used for it. Example: 'Hello' + ' World' gives 'Hello World'.

---

**Q34. What is a break statement and how does it affect a loop?**

**Answer:** break immediately terminates the loop it is inside and transfers control to the code after the loop. It is used when you want to exit a loop based on a condition before it finishes naturally.

---

**Q35. What is a continue statement and how is it different from break?**

**Answer:** continue skips the rest of the current iteration and jumps to the next one. Unlike break, it does not exit the loop — it just skips the remaining code for that iteration.

---

**Q36. What is a pass statement and when would you use it in code?**

**Answer:** pass is a null statement that does nothing. It is used as a placeholder when a block of code is syntactically required but you have nothing to write yet, like an empty class or function.

---

**Q37. What is an exception and what is the difference between a syntax error and a runtime error?**

**Answer:** An exception is an error that occurs during program execution. A syntax error is caught before running due to incorrect code structure. A runtime error occurs while the program is running, like dividing by zero.

---

**Q38. What is try/except and how does it handle exceptions in Python?**

**Answer:** try contains the code that might raise an exception, and except contains the code to run if that exception occurs. It prevents the program from crashing by gracefully handling errors.

---

**Q39. What is the import statement and what is the difference between import and from import?**

**Answer:** import module loads the entire module and you access it with module.name. from module import name imports a specific function or class directly, so you can use it without the module prefix.

---

**Q40. What is OOP and what are its four main principles?**

**Answer:** OOP (Object Oriented Programming) is a programming paradigm based on objects and classes. Its four principles are Encapsulation, Abstraction, Inheritance, and Polymorphism.

---

**Q41. What is a class and how is it different from an object?**

**Answer:** A class is a blueprint that defines attributes and behaviors. An object is an actual instance created from that class. Class is the template; object is the real thing built from it.

---

**Q42. What is an object and how is it created from a class?**

**Answer:** An object is an instance of a class that holds actual data. It is created by calling the class like a function: obj = ClassName(). Each object has its own copy of the class attributes.

---

**Q43. What is inheritance and what problem does it solve in OOP?**

**Answer:** Inheritance allows a child class to reuse the attributes and methods of a parent class. It solves code duplication by letting you extend existing functionality without rewriting it.

---

**Q44. What is __init__ and when is it automatically called in Python?**

**Answer:** __init__ is the constructor method of a class. It is automatically called when a new object is created from the class, and it is used to initialize the object's attributes.

---

**Q45. What is self and why is it the first parameter in every instance method?**

**Answer:** self refers to the current instance of the class. It is the first parameter in instance methods so Python knows which object's data to access or modify when the method is called.

---

**Q46. What is encapsulation and how is it achieved in Python?**

**Answer:** Encapsulation is hiding internal data and restricting direct access to it. In Python it is achieved using single underscore _ for protected and double underscore __ for private attributes.

---

**Q47. What is polymorphism and how does Python support it?**

**Answer:** Polymorphism means the same method name behaves differently in different classes. Python supports it through method overriding in child classes and duck typing, where any object with the right method can be used.

---

**Q48. What is a list comprehension and how is it different from a regular for loop?**

**Answer:** A list comprehension creates a new list in a single line using the syntax [expr for item in iterable]. It is more concise and generally faster than writing a full for loop with append().

---

**Q49. What is a lambda function and when should it be used over a regular function?**

**Answer:** A lambda is an anonymous function defined in one line using the lambda keyword. It should be used for short, simple operations — especially as arguments to functions like map(), filter(), or sorted().

---

**Q50. What is the difference between mutable and immutable objects and give one example of each?**

**Answer:** Mutable objects can be changed after creation — example: list. Immutable objects cannot be changed after creation — example: tuple. Mutability affects how objects behave when passed to functions or copied.